

FIT5196 Assessment 2 — Task 2: Data Reshaping

Group: Group004

Dataset: `suburb_info.xlsx` — 202 Melbourne suburbs with housing, demographic, and property price attributes.

Objective: Explore the effect of different normalisation and transformation methods on the feature columns (`number_of_houses` , `number_of_units` , `population` , `aus_born_perc` , `median_income`) and the target (`median_house_price`), with the goal of preparing the data for a linear regression model.

Two criteria drive every decision in this notebook:

1. Features must be on the same scale.
2. Features must have as linear a relationship as possible with the target.

The analysis proceeds from raw EDA through transformation comparison to a final, evidence-based recommendation.

Table of Contents

- [1. Introduction](#)
- [2. Setup & Data Loading](#)
- [3. Exploratory Data Analysis](#)
 - [3.1 Schema & Basic Statistics](#)
 - [3.2 Distribution & Skewness](#)
 - [3.2.1 QQ Plot Analysis](#)
 - [3.3 Scale Comparison](#)
 - [3.4 Correlation & Linearity with Target](#)
 - [3.5 EDA Summary](#)
 - [3.6 Linear Regression Assumptions](#)
- [4. Transformation Analysis](#)
 - [4.1 Standardisation](#)

- 4.2 Min-Max Normalisation
- 4.3 Log Transformation
- 4.4 Power Transformation
- 4.5 Box-Cox Transformation
- 5. Comparison & Evaluation
 - 5.1 Linear Regression Diagnostics (Before vs After Transformation)
- 6. Final Recommendation
 - 6.1 Recommended Transformation per Column
- 7. Conclusion

1. Introduction

This notebook explores the effect of different normalisation and transformation methods on the `suburb_info.xlsx` dataset, with the goal of preparing the data for a linear regression model that predicts `median_house_price` from five feature attributes: `number_of_houses`, `number_of_units`, `population`, `aus_born_perc`, and `median_income`.

Two criteria guide every decision in this analysis. First, features must be on the same scale so that no single variable dominates the model due to its magnitude alone. Second, features should have as linear a relationship as possible with the target variable, which requires addressing distributional skewness and heteroscedasticity where present.

The notebook proceeds as follows: Section 2 loads and parses the raw data. Section 3 conducts exploratory data analysis to assess distributions, scale disparity, skewness, and feature-target relationships. Section 4 applies and evaluates five transformation approaches — standardisation, min-max normalisation, log transformation, power (square root) transformation, and Box-Cox transformation. Section 5 consolidates the comparison across all methods. Section 6 presents the final per-column recommendation with justification. Section 7 concludes.

2. Setup & Data Loading

Three columns require string parsing before they can be used as numeric features: `aus_born_perc` carries a `%` suffix, while `median_income` and `median_house_price` use a `$` prefix with comma-separated thousands. These are stripped and cast to `float` before any analysis begins.

`municipality` is excluded from the transformation analysis. It is a categorical attribute identifying the local government area of each suburb — it carries no continuous numeric signal and cannot be meaningfully scaled or transformed. Since this task focuses on preparing continuous numeric predictors for linear regression, only the five numeric feature columns are considered throughout this notebook.

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler, MinMaxScaler, PowerTransformer
from sklearn.metrics import r2_score

# Display settings
pd.set_option('display.float_format', '{:.4f}'.format)
pd.set_option('display.max_columns', None)

# Load data – suburb is the index column
raw = pd.read_excel('suburb_info.xlsx', index_col=0)

# Three columns require parsing before use:
# aus_born_perc has a % suffix; median_income and median_house_price have $ prefix and comma separators
raw['aus_born_perc'] = raw['aus_born_perc'].str.rstrip('%').astype(float)
raw['median_income'] = raw['median_income'].str.replace('$,', '', regex=True).astype(float)
raw['median_house_price'] = raw['median_house_price'].str.replace('$,', '', regex=True).astype(float)

print(f"Shape: {raw.shape}")
print(f"\nDtypes after parsing:")
print(raw.dtypes)
print(f"\nNull counts: {raw.isnull().sum().sum()}")
```

Shape: (202, 7)

Dtypes after parsing:

number_of_houses	int64
number_of_units	int64
municipality	str
aus_born_perc	float64
median_income	float64
median_house_price	float64
population	int64
dtype:	object

Null counts: 0

3. Exploratory Data Analysis

Before applying any transformation, we examine the raw data to understand distributions, scale differences, skewness, and the relationship each feature has with the target. This drives every transformation decision made in Section 4.

3.1 Schema & Basic Statistics

We examine the numeric summary for all six columns of interest to understand the range, central tendency, and spread of each variable before any transformation is applied.

```
In [2]: # Define feature columns and target – municipality is excluded (categorical, not used in the model)
features = ['number_of_houses', 'number_of_units', 'population', 'aus_born_perc', 'median_income']
target   = 'median_house_price'
cols     = features + [target]

# Numeric summary of all columns of interest
raw[cols].describe().round(2)
```

Out [2]:

	number_of_houses	number_of_units	population	aus_born_perc	median_income	median_house_price
count	202.0000	202.0000	202.0000	202.0000	202.0000	202.0000
mean	4155.7700	2480.4100	18659.3100	68.3700	1387.9400	1218816.8300
std	3400.8600	3132.6800	9604.1900	10.7600	317.8400	471680.7400
min	283.0000	60.0000	170.0000	36.0000	633.0000	507200.0000
25%	1973.5000	620.5000	12063.0000	62.0000	1182.2500	870675.0000
50%	3256.5000	1603.0000	17583.0000	70.0000	1393.0000	1134500.0000
75%	5518.5000	3089.0000	24045.0000	77.0000	1560.0000	1467550.0000
max	23338.0000	24380.0000	54005.0000	88.0000	2352.0000	2875800.0000

3.2 Distribution & Skewness

We examine the distribution shape of each column using histograms and compute skewness scores. Skewness directly informs whether a log or power transformation is needed — values beyond ± 1 indicate meaningful asymmetry that can harm linear regression assumptions.

```
In [3]: # Compute skewness for all columns of interest
skewness = raw[cols].skew().round(4)
print("Skewness:")
print(skewness.to_string())

# Plot distributions
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
axes = axes.flatten()

for i, col in enumerate(cols):
    axes[i].hist(raw[col], bins=30, edgecolor='white', color='steelblue')
    axes[i].set_title(f'{col}\nskewness = {skewness[col]:.2f}', fontsize=11)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Frequency')

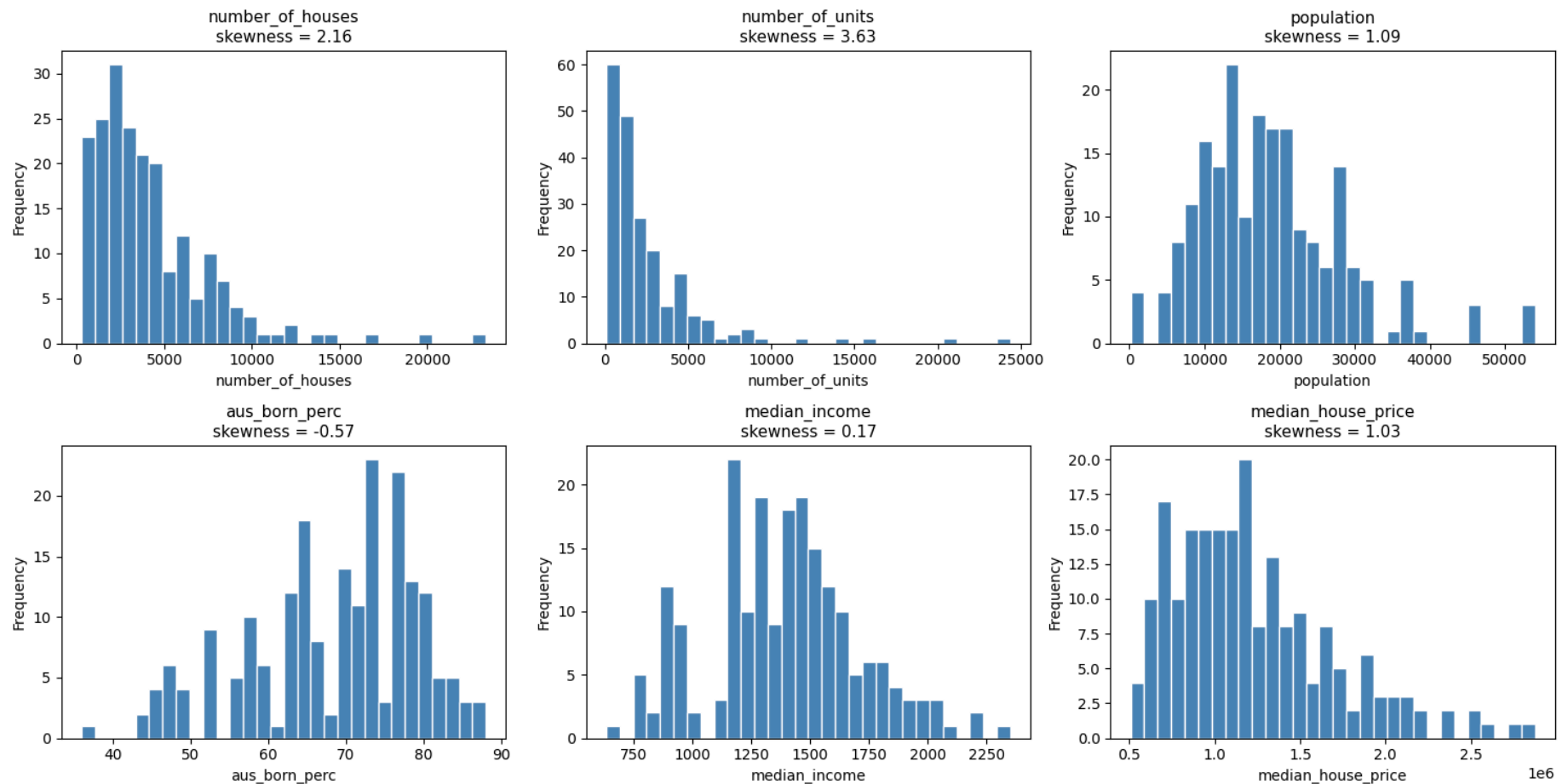
plt.suptitle('Raw Data – Distribution of All Columns', fontsize=13, y=1.01)
```

```
plt.tight_layout()
plt.show()
```

Skewness:

number_of_houses	2.1562
number_of_units	3.6302
population	1.0859
aus_born_perc	-0.5679
median_income	0.1725
median_house_price	1.0333

Raw Data — Distribution of All Columns



`number_of_units` (3.63) and `number_of_houses` (2.16) are heavily right-skewed, indicating a small number of suburbs with disproportionately high values pulling the distribution right. `population` (1.09) and `median_house_price` (1.03) are moderately right-skewed. These four columns will likely benefit from a log or power transformation to reduce asymmetry. `aus_born_perc` (-0.57) is mildly left-skewed and near-symmetric enough that shape correction is unlikely to be necessary. `median_income` (0.17) is essentially symmetric and requires no distributional transformation.

3.2.1 QQ Plot Analysis

QQ plots provide a visual assessment of whether each variable's distribution approximates normality. Points falling along the diagonal reference line indicate normality; systematic deviations — curves, S-shapes, or heavy tails — confirm the skewness identified numerically above.

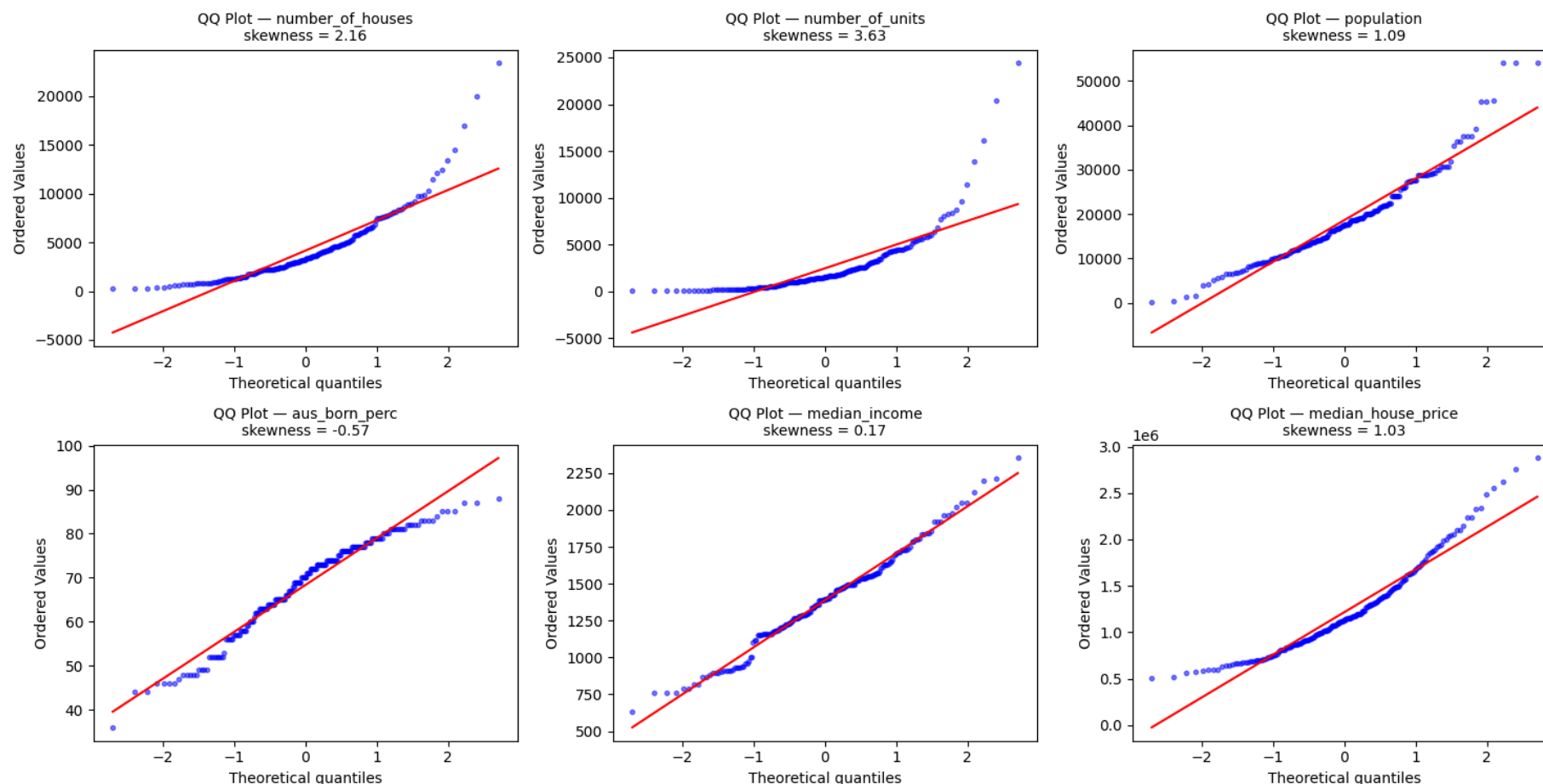
```
In [4]: from scipy.stats import probplot

fig, axes = plt.subplots(2, 3, figsize=(15, 8))
axes = axes.flatten()

for i, col in enumerate(cols):
    probplot(raw[col], dist="norm", plot=axes[i])
    axes[i].set_title(f'QQ Plot - {col}\nskewness = {raw[col].skew():.2f}', fontsize=10)
    axes[i].get_lines()[0].set(markersize=3, alpha=0.5) # smaller points, less clutter

plt.suptitle('Raw Data - QQ Plots (Normality Assessment)', fontsize=13, y=1.01)
plt.tight_layout()
plt.show()
```

Raw Data — QQ Plots (Normality Assessment)



The QQ plots confirm the skewness findings from Section 3.2 visually. `number_of_units` (skewness 3.63) and `number_of_houses` (2.16) show the strongest departures from normality: points deviate below the reference line in the lower tail and curve sharply above it in the upper tail, the classic QQ signature of heavy right skew. `population` (1.09) and `median_house_price` (1.03) show moderate upper-tail deviation, consistent with their intermediate skewness. `aus_born_perc` (-0.57) displays a mild S-curve pattern — lower-tail points fall below the line — reflecting its slight left skew, though the overall departure is small. `median_income` (0.17) tracks the reference line most closely of all columns, confirming it is effectively symmetric. These patterns directly motivate the non-linear transformations applied in Section 4.

3.3 Scale Comparison

The numeric summary already hints at severe scale disparity across features. Here we make this explicit by comparing the ranges and standard deviations directly. Features on incompatible scales will cause a linear regression model to weight high-magnitude variables disproportionately, regardless of their actual predictive relevance.

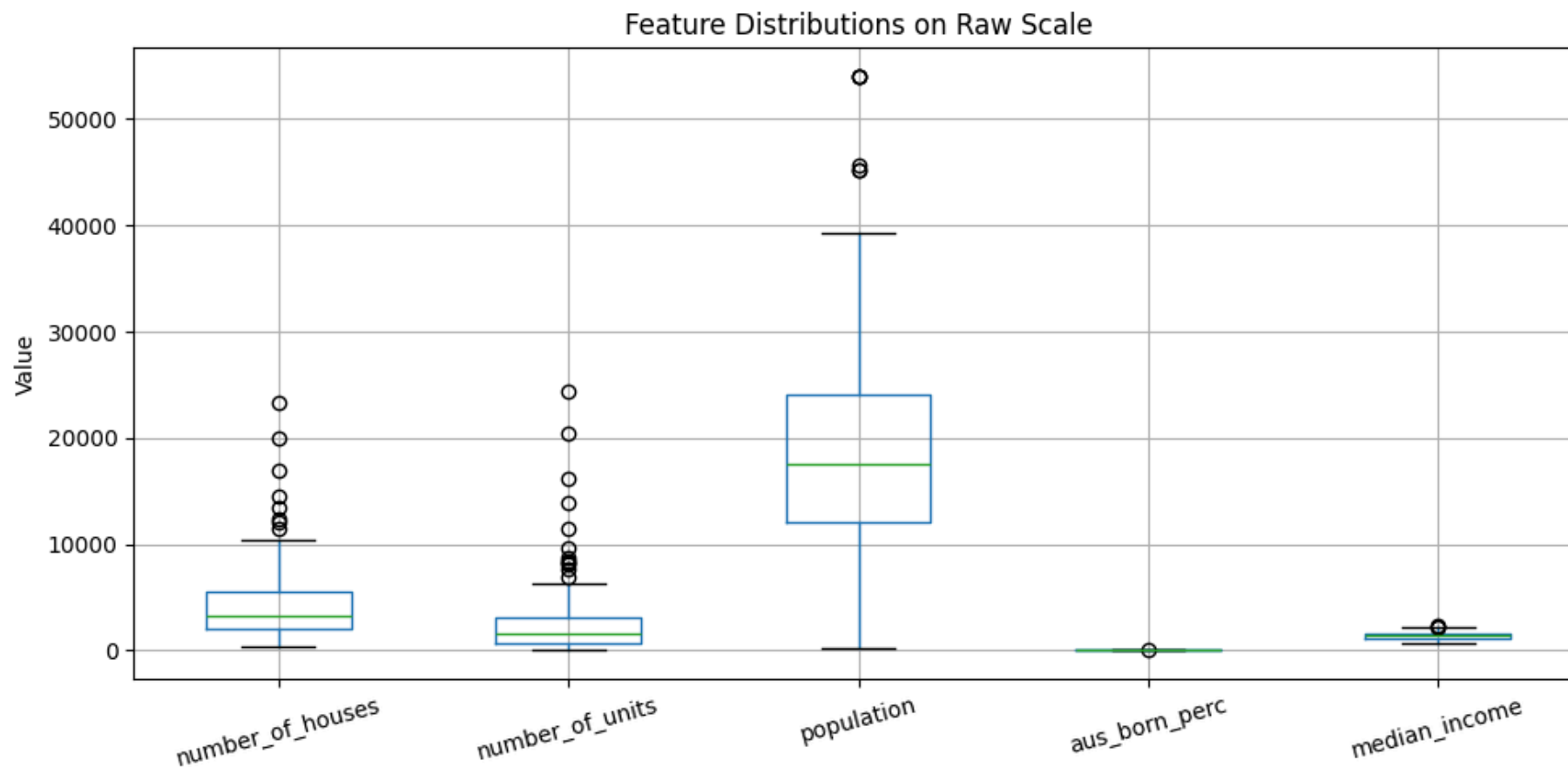
```
In [5]: # Summarise scale differences across features
scale_summary = pd.DataFrame({
    'min': raw[features].min(),
    'max': raw[features].max(),
    'range': raw[features].max() - raw[features].min(),
    'std': raw[features].std()
}).round(2)

print("Scale comparison across features:")
print(scale_summary.to_string())

# Visualise using boxplots on the raw scale – differences should be stark
fig, ax = plt.subplots(figsize=(10, 5))
raw[features].boxplot(ax=ax)
ax.set_title('Feature Distributions on Raw Scale', fontsize=12)
ax.set_ylabel('Value')
ax.set_xticklabels(features, rotation=15)
plt.tight_layout()
plt.show()
```

Scale comparison across features:

	min	max	range	std
number_of_houses	283.0000	23338.0000	23055.0000	3400.8600
number_of_units	60.0000	24380.0000	24320.0000	3132.6800
population	170.0000	54005.0000	53835.0000	9604.1900
aus_born_perc	36.0000	88.0000	52.0000	10.7600
median_income	633.0000	2352.0000	1719.0000	317.8400



The scale disparity across features is severe. `population` spans a range of 53,835 with a standard deviation of 9,604 — roughly 900 times larger than `aus_born_perc`, which spans just 52 with a standard deviation of 10.76. `number_of_houses` and `number_of_units` both range to approximately 23,000–24,000, while `median_income` sits in the hundreds. `aus_born_perc` is completely invisible on the shared boxplot axis — its entire range is compressed into a flat line at the bottom. Without scaling, a linear regression model will assign disproportionate weight to high-magnitude features like `population` purely due to their numeric scale, not their predictive relevance. All five features require a scaling step before modelling.

3.4 Correlation & Linearity with Target

We examine the Pearson correlation between each feature and `median_house_price`, then plot scatter plots to assess whether the relationships are actually linear. A high correlation is only useful to a linear model if the underlying relationship is linear — a curved or heteroscedastic relationship will violate regression assumptions even if the correlation coefficient looks reasonable.

```
In [6]: # Pearson correlation of each feature with the target
correlations = raw[features].corrwith(raw[target]).round(4).sort_values(ascending=False)
print("Pearson correlation with median_house_price:")
print(correlations.to_string())

# Scatter plots of each feature against the target
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
axes = axes.flatten()

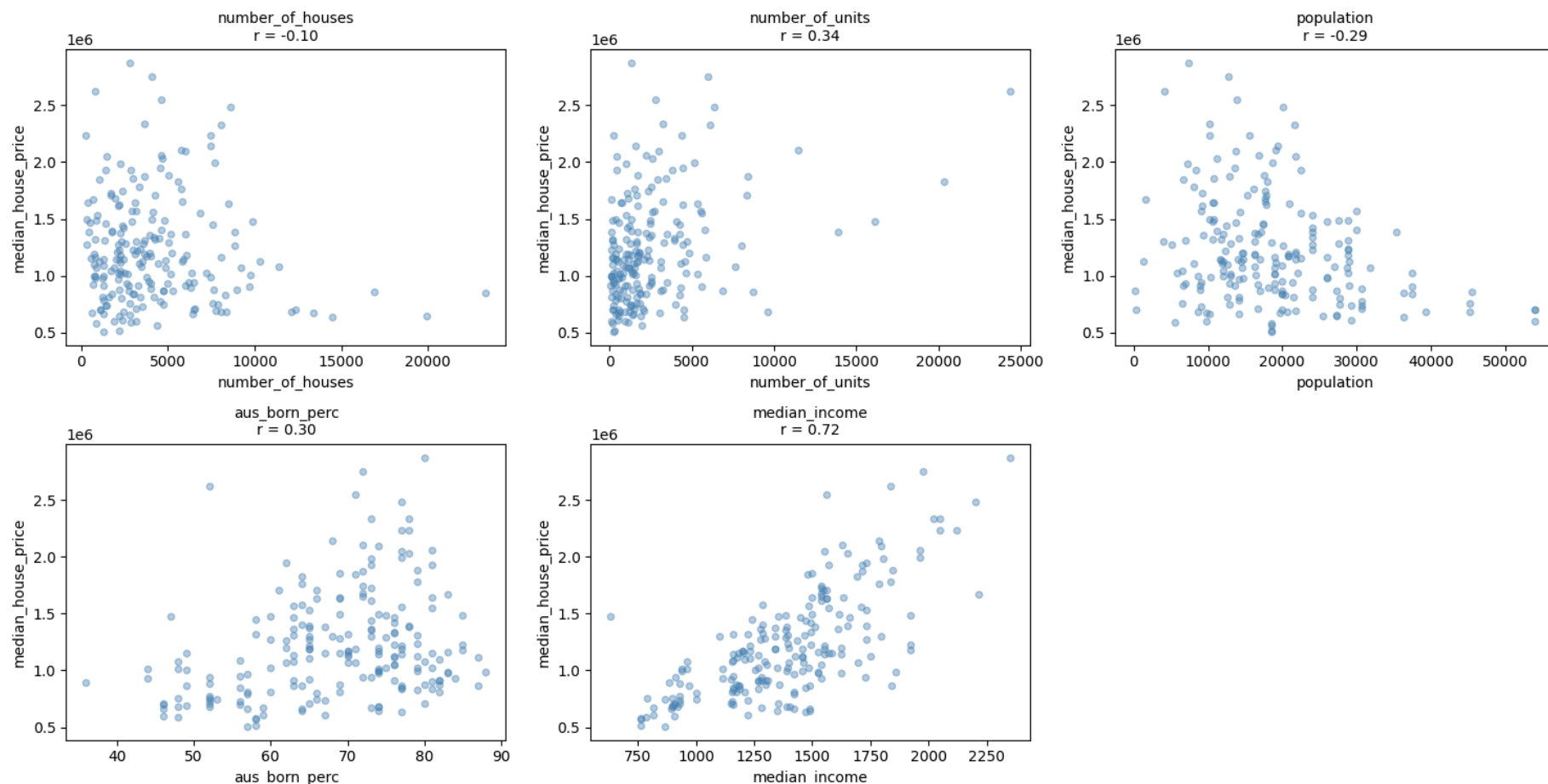
for i, col in enumerate(features):
    axes[i].scatter(raw[col], raw[target], alpha=0.4, color='steelblue', s=20)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('median_house_price')
    r = correlations[col]
    axes[i].set_title(f'{col}\nr = {r:.2f}', fontsize=10)

# Hide the unused sixth subplot
axes[5].set_visible(False)

plt.suptitle('Raw Features vs. median_house_price', fontsize=13, y=1.01)
plt.tight_layout()
plt.show()
```

```
Pearson correlation with median_house_price:
median_income      0.7194
number_of_units    0.3439
aus_born_perc      0.3039
number_of_houses   -0.1045
population          -0.2872
```

Raw Features vs. median_house_price



`median_income` is the strongest predictor ($r = 0.72$), but the scatter plot shows variance increasing at higher income values — a classic sign of heteroscedasticity. `number_of_units` ($r = 0.34$) and `aus_born_perc` ($r = 0.30$) have weak positive relationships with the target. `population` ($r = -0.29$) and `number_of_houses` ($r = -0.10$) are negatively correlated, with `number_of_houses` showing virtually no linear signal. The fan-shaped scatter in `number_of_units` and `population` confirms that their right skew is directly producing non-constant variance — transforming these columns should improve both linearity and homoscedasticity.

3.5 EDA Summary

The raw data reveals two distinct problems that must be addressed before fitting a linear regression model:

Scale disparity: Features operate on completely incompatible scales — `population` ranges up to 54,005 while `aus_born_perc` ranges from 36 to 88. Without scaling, a linear model will assign disproportionate weight to high-magnitude features regardless of their actual predictive relevance. All features require scaling.

Skewness: `number_of_units` (3.63) and `number_of_houses` (2.16) are heavily right-skewed; `population` (1.09) and `median_house_price` (1.03) are moderately right-skewed. Right-skewed features produce heteroscedastic scatter plots where variance fans out at higher values — visible in the plots above for `number_of_units` and `population`. This non-constant variance violates the linearity assumption and will weaken the model. These columns require a variance-stabilising transformation before or alongside scaling.

`aus_born_perc` (-0.57) and `median_income` (0.17) are near-symmetric and primarily need scaling rather than shape correction.

These findings motivate the transformation analysis in Section 4.

3.6 Linear Regression Assumptions

Before selecting transformations, it is worth stating the key assumptions of ordinary least squares (OLS) linear regression that the data preparation aims to satisfy:

1. **Linearity:** The relationship between each feature and the target should be approximately linear. The scatter plots in Section 3.4 show that several features have non-linear or fan-shaped relationships with `median_house_price`, suggesting transformation may help.
2. **Homoscedasticity (constant variance):** Residuals should have constant variance across the range of predicted values. The fan-shaped scatter for `median_income` and `number_of_units` against the target indicates heteroscedasticity — variance increases with the predictor value. This is a direct consequence of right-skewed distributions and is a strong signal that a variance-stabilising transformation is needed.
3. **Normality of residuals:** Residuals should be approximately normally distributed. Right-skewed features tend to produce right-skewed residuals. Correcting feature skewness before modelling helps satisfy this assumption.
4. **No multicollinearity:** Features should not be highly correlated with each other. This is a model-selection concern rather than a transformation concern, so we note it but do not address it further in this notebook.

The transformations evaluated in Section 4 primarily target assumptions 1–3. Scale normalisation (standardisation, min-max) addresses coefficient interpretability but does not improve any of these assumptions. Non-linear transformations (log, square root, Box-Cox) can improve all three by reducing skewness and stabilising variance.

4. Transformation Analysis

The EDA established two problems: incompatible scales and right-skewed distributions. We now apply five transformation approaches — standardisation, min-max normalisation, log transformation, power (square root) transformation, and Box-Cox transformation — and evaluate each against both criteria. For each method we assess the resulting distributions, scale consistency, and correlation with `median_house_price`.

4.1 Standardisation

Standardisation (z-score normalisation) rescales each column to zero mean and unit variance by subtracting the mean and dividing by the standard deviation. It addresses scale disparity but does not change the shape of the distribution — a right-skewed column will remain right-skewed after standardisation. We apply it to all five features and evaluate.

```
In [7]: # Apply z-score standardisation to all features
scaler_std = StandardScaler()
std_scaled = pd.DataFrame(
    scaler_std.fit_transform(raw[features]),
    columns=features,
    index=raw.index
)

# Check resulting distributions
print("Standardised features – mean and std:")
print(pd.DataFrame({
    'mean': std_scaled.mean().round(6),
    'std': std_scaled.std().round(4),
    'skew': std_scaled.skew().round(4)
}).to_string())

# Correlation with target after standardisation
corr_std = std_scaled.corrwith(raw[target]).round(4).sort_values(ascending=False)
```

```

print("\nCorrelation with median_house_price after standardisation:")
print(corr_std.to_string())

# Plot distributions after standardisation
fig, axes = plt.subplots(1, 5, figsize=(18, 4))
for i, col in enumerate(features):
    axes[i].hist(std_scaled[col], bins=30, edgecolor='white', color='steelblue')
    axes[i].set_title(f'{col}\nskew = {std_scaled[col].skew():.2f}', fontsize=9)
    axes[i].set_xlabel('Standardised value')

plt.suptitle('Standardised Features – Distributions', fontsize=12, y=1.02)
plt.tight_layout()
plt.show()

```

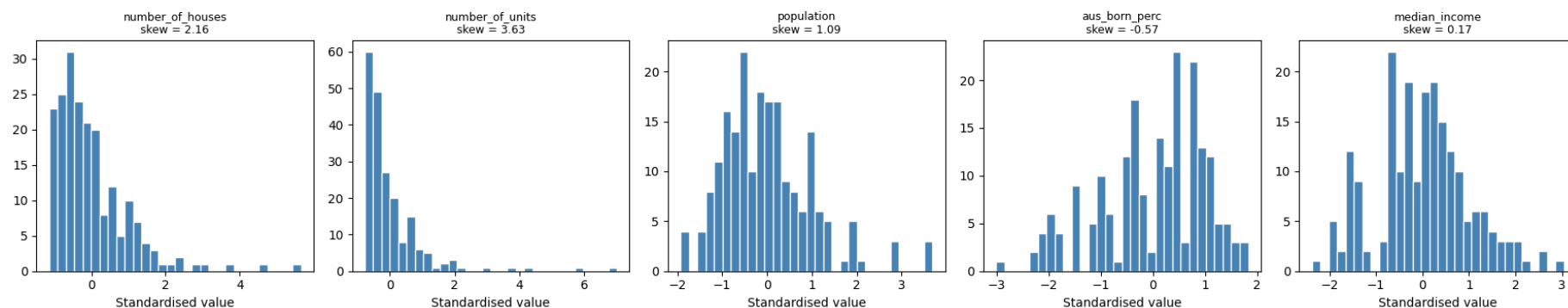
Standardised features – mean and std:

	mean	std	skew
number_of_houses	-0.0000	1.0025	2.1562
number_of_units	-0.0000	1.0025	3.6302
population	0.0000	1.0025	1.0859
aus_born_perc	0.0000	1.0025	-0.5679
median_income	-0.0000	1.0025	0.1725

Correlation with median_house_price after standardisation:

median_income	0.7194
number_of_units	0.3439
aus_born_perc	0.3039
number_of_houses	-0.1045
population	-0.2872

Standardised Features — Distributions



Standardisation brings all features to the same scale (mean = 0, std = 1), which resolves the scale disparity problem. However, the skewness values are completely unchanged — `number_of_units` remains at 3.63 and `number_of_houses` at 2.16, identical to the raw data. This is expected: standardisation is a linear transformation and cannot alter distribution shape. Correlations with `median_house_price` are also identical to the raw correlations, which confirms that linear rescaling has no effect on linear relationships. Standardisation alone is insufficient for the skewed columns.

4.2 Min-Max Normalisation

Min-max normalisation rescales each column to the [0, 1] range by subtracting the minimum and dividing by the range. Like standardisation, it is a linear transformation — it resolves scale disparity but leaves skewness and correlation with the target completely unchanged.

```
In [8]: # Apply min-max normalisation to all features
scaler_mm = MinMaxScaler()
mm_scaled = pd.DataFrame(
    scaler_mm.fit_transform(raw[features]),
    columns=features,
    index=raw.index
)

# Check resulting distributions
print("Min-max normalised features — range, mean, skew:")
print(pd.DataFrame({
    'min': mm_scaled.min().round(4),
    'max': mm_scaled.max().round(4),
    'mean': mm_scaled.mean().round(4),
    'skew': mm_scaled.skew().round(4)
}).to_string())

# Correlation with target
corr_mm = mm_scaled.corrwith(raw[target]).round(4).sort_values(ascending=False)
print("\nCorrelation with median_house_price after min-max normalisation:")
print(corr_mm.to_string())

# Plot distributions
fig, axes = plt.subplots(1, 5, figsize=(18, 4))
for i, col in enumerate(features):
```



```

axes[i].hist(mm_scaled[col], bins=30, edgecolor='white', color='steelblue')
axes[i].set_title(f'{col}\nskew = {mm_scaled[col].skew():.2f}', fontsize=9)
axes[i].set_xlabel('Normalised value')

plt.suptitle('Min-Max Normalised Features – Distributions', fontsize=12, y=1.02)
plt.tight_layout()
plt.show()

```

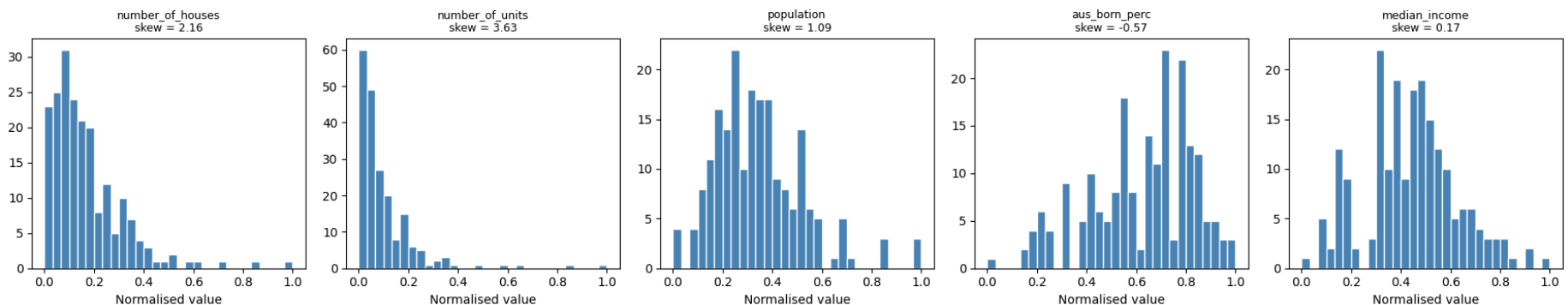
Min-max normalised features – range, mean, skew:

	min	max	mean	skew
number_of_houses	0.0000	1.0000	0.1680	2.1562
number_of_units	0.0000	1.0000	0.0995	3.6302
population	0.0000	1.0000	0.3434	1.0859
aus_born_perc	0.0000	1.0000	0.6224	-0.5679
median_income	0.0000	1.0000	0.4392	0.1725

Correlation with median_house_price after min-max normalisation:

median_income	0.7194
number_of_units	0.3439
aus_born_perc	0.3039
number_of_houses	-0.1045
population	-0.2872

Min-Max Normalised Features — Distributions



As expected, min-max normalisation compresses all features into the [0, 1] range, resolving scale disparity. However, identical to standardisation, the skewness values and correlations with `median_house_price` are completely unchanged from the raw data. Both standardisation and min-max normalisation are linear transformations — they solve the scale problem but do nothing for distributional

shape. The heavily skewed columns `number_of_units` and `number_of_houses` will still violate regression assumptions. A non-linear transformation is required to address skewness.

4.3 Log Transformation

Log transformation is a non-linear variance-stabilising transformation that compresses large values and spreads small ones, directly reducing right skewness. It is applied only to columns with meaningful right skew — `number_of_houses`, `number_of_units`, `population`, and `median_house_price`. `aus_born_perc` and `median_income` are near-symmetric and do not require shape correction. We also apply log to the target to assess whether it improves the linearity of feature-target relationships.

```
In [9]: # Apply log transformation to right-skewed columns only
# aus_born_perc and median_income are excluded – near-symmetric, no shape correction needed
log_cols = ['number_of_houses', 'number_of_units', 'population']
log_transformed = raw[features].copy()

for col in log_cols:
    log_transformed[col] = np.log(raw[col])

# Also log-transform the target for linearity assessment
log_target = np.log(raw[target])

# Skewness after log transformation
print("Skewness after log transformation:")
print(pd.DataFrame({
    'skew_raw': raw[features].skew().round(4),
    'skew_log': log_transformed.skew().round(4)
}).to_string())

# Correlation with log-transformed target
corr_log = log_transformed.corrwith(log_target).round(4).sort_values(ascending=False)
print("\nCorrelation with log(median_house_price) after log transformation:")
print(corr_log.to_string())

# Plot distributions after log transformation
fig, axes = plt.subplots(1, 5, figsize=(18, 4))
for i, col in enumerate(features):
    axes[i].hist(log_transformed[col], bins=30, edgecolor='white', color='steelblue')
```

```

    axes[i].set_title(f'{col}\nskew = {log_transformed[col].skew():.2f}', fontsize=9)
    axes[i].set_xlabel('Log value' if col in log_cols else 'Raw value')

plt.suptitle('Log-Transformed Features – Distributions', fontsize=12, y=1.02)
plt.tight_layout()
plt.show()

# Scatter plots against log target
fig, axes = plt.subplots(1, 5, figsize=(18, 4))
for i, col in enumerate(features):
    axes[i].scatter(log_transformed[col], log_target, alpha=0.4, color='steelblue', s=20)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('log(median_house_price)')
    axes[i].set_title(f'r = {corr_log[col]:.2f}', fontsize=10)

plt.suptitle('Log-Transformed Features vs. log(median_house_price)', fontsize=12, y=1.02)
plt.tight_layout()
plt.show()

```

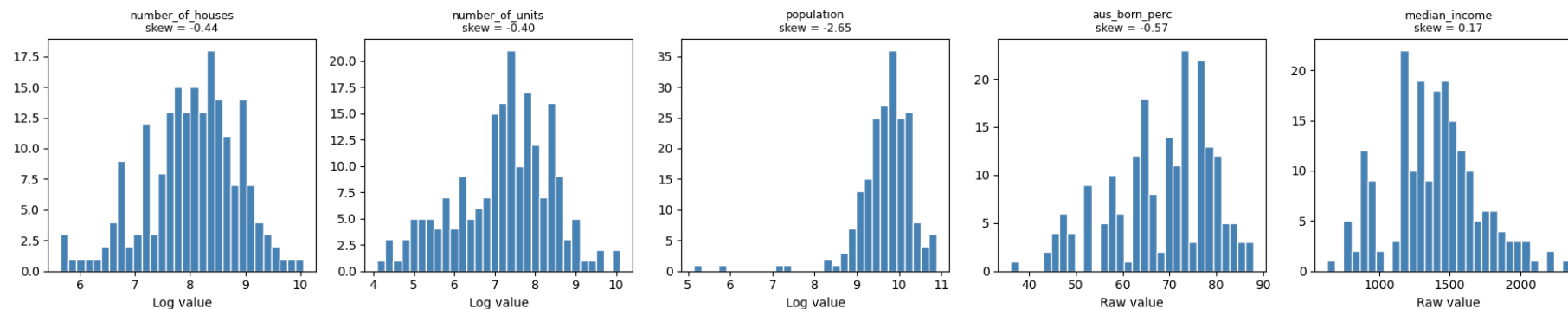
Skewness after log transformation:

	skew_raw	skew_log
number_of_houses	2.1562	-0.4425
number_of_units	3.6302	-0.3994
population	1.0859	-2.6451
aus_born_perc	-0.5679	-0.5679
median_income	0.1725	0.1725

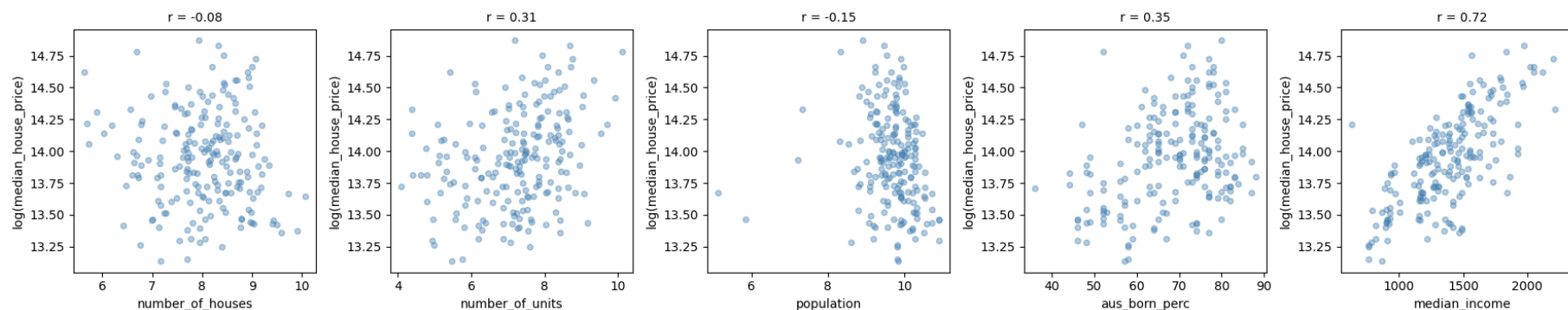
Correlation with log(median_house_price) after log transformation:

median_income	0.7198
aus_born_perc	0.3472
number_of_units	0.3097
number_of_houses	-0.0788
population	-0.1531

Log-Transformed Features — Distributions



Log-Transformed Features vs. log(median_house_price)



Log transformation substantially reduces skewness in `number_of_houses` (2.16 $\rightarrow$ -0.44) and `number_of_units` (3.63 $\rightarrow$ -0.40), bringing both close to symmetry. However, `population` is overcorrected from 1.09 to -2.65 — the log transformation is too aggressive for this column and introduces a new problem in the opposite direction. `aus_born_perc` and `median_income` are unchanged as expected. Correlations with the log-transformed target show marginal improvement overall, suggesting the feature-target relationships benefit slightly from log-transforming the target. A more flexible transformation method is needed for `population` — Box-Cox, which optimises the transformation parameter rather than fixing it at log ($\lambda=0$), is a better candidate.

4.4 Power Transformation

Power transformations apply a fixed exponent to each column — common choices include square root ($x^{0.5}$), square (x^2), and reciprocal ($1/x$). Unlike Box-Cox, the exponent is chosen manually rather than optimised. Square root is a natural candidate for right-

skewed columns because it compresses large values more gently than log, making it useful when log overcorrects (as observed for `population` in Section 4.3).

We apply square root to the three right-skewed columns and leave `aus_born_perc` and `median_income` unchanged, consistent with the log transformation approach.

```
In [10]: # Apply square root transformation to right-skewed columns
sqrt_cols = ['number_of_houses', 'number_of_units', 'population']
sqrt_transformed = raw[features].copy()

for col in sqrt_cols:
    sqrt_transformed[col] = np.sqrt(raw[col])

# Also square-root transform the target for comparison
sqrt_target = np.sqrt(raw[target])

# Skewness after square root
print("Skewness after square root transformation:")
print(pd.DataFrame({
    'skew_raw': raw[features].skew().round(4),
    'skew_sqrt': sqrt_transformed.skew().round(4)
}).to_string())

# Correlation with sqrt-transformed target
corr_sqrt = sqrt_transformed.corrwith(sqrt_target).round(4).sort_values(ascending=False)
print("\nCorrelation with sqrt(median_house_price) after square root transformation:")
print(corr_sqrt.to_string())

# Plot distributions after square root
fig, axes = plt.subplots(1, 5, figsize=(18, 4))
for i, col in enumerate(features):
    axes[i].hist(sqrt_transformed[col], bins=30, edgecolor='white', color='steelblue')
    axes[i].set_title(f'{col}\nskew = {sqrt_transformed[col].skew():.2f}', fontsize=9)
    axes[i].set_xlabel('Sqrt value' if col in sqrt_cols else 'Raw value')

plt.suptitle('Square Root Transformed Features – Distributions', fontsize=12, y=1.02)
plt.tight_layout()
plt.show()
```

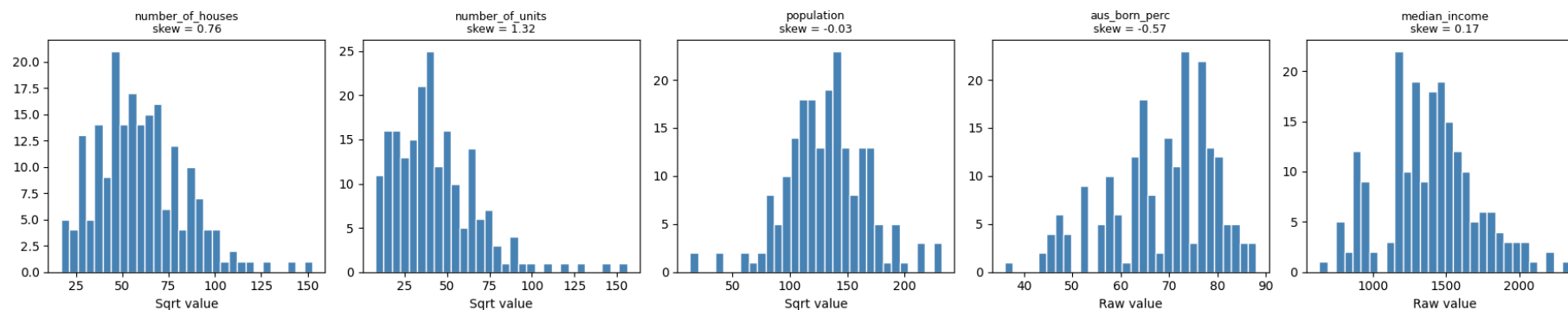
Skewness after square root transformation:

	skew_raw	skew_sqrt
number_of_houses	2.1562	0.7625
number_of_units	3.6302	1.3239
population	1.0859	-0.0288
aus_born_perc	-0.5679	-0.5679
median_income	0.1725	0.1725

Correlation with $\sqrt{\text{median_house_price}}$ after square root transformation:

median_income	0.7235
number_of_units	0.3499
aus_born_perc	0.3269
number_of_houses	-0.0889
population	-0.2538

Square Root Transformed Features — Distributions



Square root transformation reduces skewness in the right-skewed columns but less aggressively than log. For `population`, square root corrects skewness from 1.09 to -0.03 — near-perfect symmetry — whereas log overcorrected to -2.65 . For `number_of_houses`, skewness drops from 2.16 to 0.76, a meaningful improvement but still noticeably right-skewed. `number_of_units` remains the most problematic column at 1.32, down from 3.63 but still well above the ± 0.5 threshold for approximate symmetry. The fixed exponent (0.5) cannot adapt to different columns, so it overcorrects some and undercorrects others. Box-Cox generalises this by optimising the exponent (λ) for each column individually, making it strictly more flexible.

4.5 Box-Cox Transformation

Box-Cox transformation generalises both log ($\lambda=0$) and power transformations ($\lambda \neq 0$) by finding the optimal lambda for each column that best normalises the distribution. This makes it more flexible than any fixed-exponent approach — particularly useful for columns like **population** where log overcorrected and square root may undercorrect. Box-Cox requires strictly positive values, which all columns satisfy.

```
In [11]: # Apply Box-Cox transformation to all features
# PowerTransformer with method='box-cox' requires strictly positive values – confirmed for all columns
pt = PowerTransformer(method='box-cox')
boxcox_transformed = pd.DataFrame(
    pt.fit_transform(raw[features]),
    columns=features,
    index=raw.index
)

# Also Box-Cox transform the target
pt_target = PowerTransformer(method='box-cox')
boxcox_target = pt_target.fit_transform(raw[[target]]).flatten()

# Report optimal lambda values found per column
print("Optimal Box-Cox lambda values:")
for col, lam in zip(features, pt.lambdas_):
    print(f" {col:<25}: lambda = {lam:.4f}")

# Skewness comparison
print("\nSkewness comparison:")
print(pd.DataFrame({
    'skew_raw': raw[features].skew().round(4),
    'skew_log': log_transformed.skew().round(4),
    'skew_boxcox': boxcox_transformed.skew().round(4)
}).to_string())

# Correlation with Box-Cox transformed target
corr_bc = boxcox_transformed.corrwith(pd.Series(boxcox_target, index=raw.index)).round(4).sort_values(ascending=False)
print("\nCorrelation with BoxCox(median_house_price) after Box-Cox transformation:")
print(corr_bc.to_string())

# Plot distributions after Box-Cox
fig, axes = plt.subplots(1, 5, figsize=(18, 4))
for i, col in enumerate(features):
```

```

    axes[i].hist(boxcox_transformed[col], bins=30, edgecolor='white', color='steelblue')
    axes[i].set_title(f'{col}\nskew = {boxcox_transformed[col].skew():.2f}', fontsize=9)
    axes[i].set_xlabel('Box-Cox value')

plt.suptitle('Box-Cox Transformed Features – Distributions', fontsize=12, y=1.02)
plt.tight_layout()
plt.show()

# Scatter plots against Box-Cox target
fig, axes = plt.subplots(1, 5, figsize=(18, 4))
for i, col in enumerate(features):
    axes[i].scatter(boxcox_transformed[col], boxcox_target, alpha=0.4, color='steelblue', s=20)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('BoxCox(median_house_price)')
    axes[i].set_title(f'r = {corr_bc[col]:.2f}', fontsize=10)

plt.suptitle('Box-Cox Transformed Features vs. BoxCox(median_house_price)', fontsize=12, y=1.02)
plt.tight_layout()
plt.show()

```

Optimal Box-Cox lambda values:

number_of_houses	: lambda = 0.1828
number_of_units	: lambda = 0.1269
population	: lambda = 0.5458
aus_born_perc	: lambda = 2.4607
median_income	: lambda = 0.7332

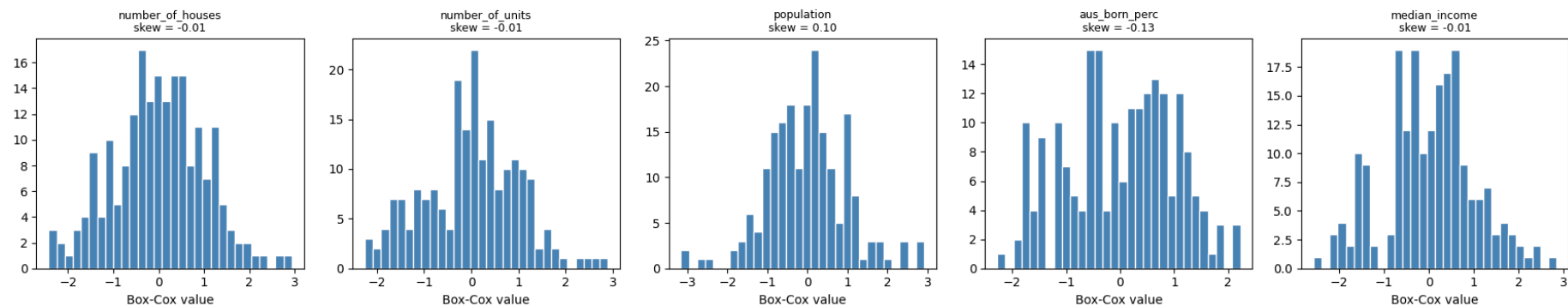
Skewness comparison:

	skew_raw	skew_log	skew_boxcox
number_of_houses	2.1562	-0.4425	-0.0064
number_of_units	3.6302	-0.3994	-0.0141
population	1.0859	-2.6451	0.1005
aus_born_perc	-0.5679	-0.5679	-0.1286
median_income	0.1725	0.1725	-0.0084

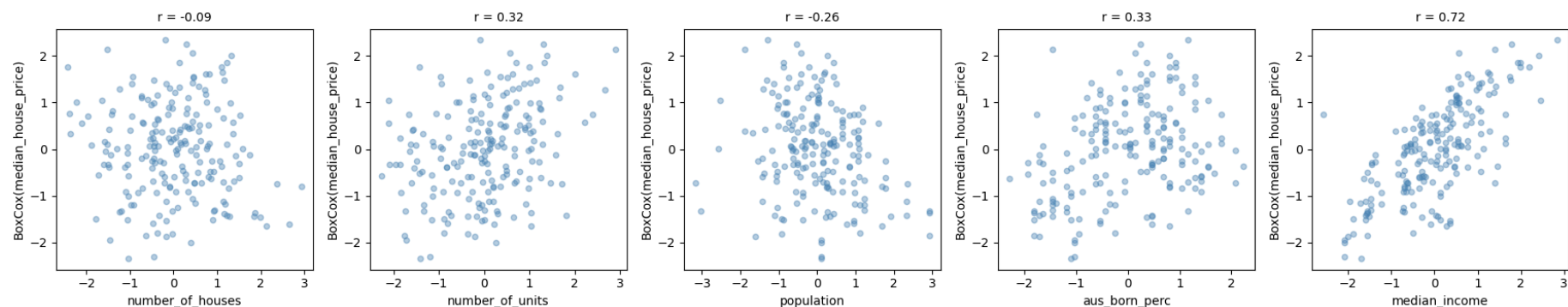
Correlation with BoxCox(median_house_price) after Box-Cox transformation:

median_income	0.7151
aus_born_perc	0.3277
number_of_units	0.3197
number_of_houses	-0.0870
population	-0.2595

Box-Cox Transformed Features — Distributions



Box-Cox Transformed Features vs. BoxCox(median_house_price)



Box-Cox transformation achieves near-perfect symmetry across all five features — every column lands within ± 0.13 skewness of zero. The improvement over log transformation is most dramatic for **population**: log overcorrected it to -2.65, while Box-Cox corrects it to 0.10 using an optimal $\lambda=0.55$ (approximately a square root transformation). For **number_of_houses** ($\lambda=0.18$) and **number_of_units** ($\lambda=0.13$), the optimal lambda is close to zero, consistent with the near-log behaviour observed in Section 4.3. Box-Cox also corrects **aus_born_perc** ($\lambda=2.46$) and **median_income** ($\lambda=0.73$), which were not meaningfully skewed to begin with — confirming it handles near-symmetric columns without distorting them. Correlations with the Box-Cox transformed target remain consistent with previous methods, indicating that the feature-target relationships are stable across transformations.

5. Comparison & Evaluation

We now consolidate the findings from all five transformation methods into a direct comparison across the two criteria established in Section 1: scale consistency and linearity with the target. We evaluate each method on skewness reduction and correlation improvement to identify the best approach for each column.

```
In [12]: # Consolidated skewness comparison across all methods
skew_comparison = pd.DataFrame({
    'Raw': raw[features].skew().round(4),
    'Standardised': std_scaled.skew().round(4),
    'Min-Max': mm_scaled.skew().round(4),
    'Log': log_transformed.skew().round(4),
    'Sqrt': sqrt_transformed.skew().round(4),
    'Box-Cox': boxcox_transformed.skew().round(4)
})
print("Skewness across all transformation methods:")
print(skew_comparison.to_string())

# Consolidated correlation comparison
corr_comparison = pd.DataFrame({
    'Raw': raw[features].corrwith(raw[target]).round(4),
    'Standardised': std_scaled.corrwith(raw[target]).round(4),
    'Min-Max': mm_scaled.corrwith(raw[target]).round(4),
    'Log': log_transformed.corrwith(log_target).round(4),
    'Sqrt': sqrt_transformed.corrwith(sqrt_target).round(4),
    'Box-Cox': boxcox_transformed.corrwith(pd.Series(boxcox_target, index=raw.index)).round(4)
})
print("\nCorrelation with target across all transformation methods:")
print(corr_comparison.to_string())

# Visualise skewness comparison as a heatmap
fig, axes = plt.subplots(1, 2, figsize=(18, 5))

sns.heatmap(
    skew_comparison,
    annot=True, fmt='.2f', cmap='RdYlGn_r',
    center=0, vmin=-3, vmax=3,
    linewidths=0.5, ax=axes[0]
)
axes[0].set_title('Skewness by Transformation Method\n(closer to 0 is better)', fontsize=11)
```

```

sns.heatmap(
    corr_comparison,
    annot=True, fmt='.2f', cmap='RdYlGn',
    center=0, vmin=-1, vmax=1,
    linewidths=0.5, ax=axes[1]
)
axes[1].set_title('Correlation with Target by Transformation Method\n(higher absolute value is better)', fontsize=14)

plt.tight_layout()
plt.show()

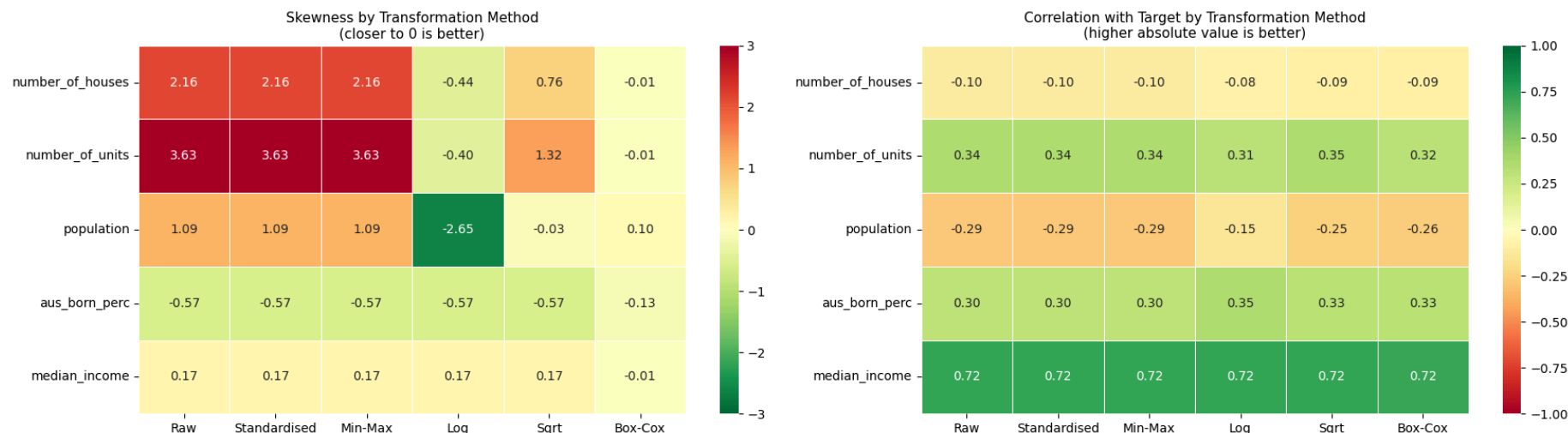
```

Skewness across all transformation methods:

	Raw	Standardised	Min-Max	Log	Sqrt	Box-Cox
number_of_houses	2.1562	2.1562	2.1562	-0.4425	0.7625	-0.0064
number_of_units	3.6302	3.6302	3.6302	-0.3994	1.3239	-0.0141
population	1.0859	1.0859	1.0859	-2.6451	-0.0288	0.1005
aus_born_perc	-0.5679	-0.5679	-0.5679	-0.5679	-0.5679	-0.1286
median_income	0.1725	0.1725	0.1725	0.1725	0.1725	-0.0084

Correlation with target across all transformation methods:

	Raw	Standardised	Min-Max	Log	Sqrt	Box-Cox
number_of_houses	-0.1045	-0.1045	-0.1045	-0.0788	-0.0889	-0.0870
number_of_units	0.3439	0.3439	0.3439	0.3097	0.3499	0.3197
population	-0.2872	-0.2872	-0.2872	-0.1531	-0.2538	-0.2595
aus_born_perc	0.3039	0.3039	0.3039	0.3472	0.3269	0.3277
median_income	0.7194	0.7194	0.7194	0.7198	0.7235	0.7151



The comparison tables and heatmaps make two conclusions unambiguous.

On skewness: Standardisation and min-max normalisation are purely linear transformations — they leave skewness completely unchanged from the raw data. Log transformation corrects `number_of_houses` and `number_of_units` well but severely overcorrects `population` (-2.65), introducing a new problem. Note that log and square root were applied only to the three right-skewed columns (`number_of_houses`, `number_of_units`, `population`) — `aus_born_perc` and `median_income` were left untransformed because they are already near-symmetric, which is why their skewness values are identical to raw in those columns. Square root sits between raw and log: it reduces `number_of_houses` from 2.16 to 0.76 and `population` from 1.09 to -0.03 (near-perfect), but undercorrects `number_of_units` (3.63 → 1.32, still meaningfully skewed). Box-Cox achieves near-perfect symmetry across all five features, with every column within ± 0.13 of zero, confirming it is the superior transformation for shape correction.

On correlation with the target: No transformation method meaningfully changes the correlation values. `median_income` remains the dominant predictor (~0.72) across all methods. Square root produces marginally higher correlations for `number_of_units` (0.35) and `median_income` (0.72), but the differences are negligible. This indicates the feature-target relationships are stable and that transformation primarily benefits the distributional assumptions of linear regression rather than directly boosting predictive signal.

Box-Cox transformation is the clear winner for skewness correction. For scale normalisation, Box-Cox already standardises the output (sklearn's PowerTransformer applies zero-mean, unit-variance scaling by default), so no additional scaling step is required when Box-Cox is used.

5.1 Linear Regression Diagnostics (Before vs After Transformation)

As a final piece of evidence, we fit a simple OLS linear regression on the raw features and on the Box-Cox transformed features to compare model fit and residual behaviour. The goal is not to build a final model but to confirm that the transformation improves the suitability of the data for linear regression — specifically, whether R^2 improves and whether residuals become more randomly distributed.

```
In [13]: # Fit linear regression on raw features vs raw target
lr_raw = LinearRegression()
lr_raw.fit(raw[features], raw[target])
pred_raw = lr_raw.predict(raw[features])
r2_raw = r2_score(raw[target], pred_raw)
resid_raw = raw[target] - pred_raw

# Fit linear regression on Box-Cox features vs Box-Cox target
lr_bc = LinearRegression()
lr_bc.fit(boxcox_transformed, boxcox_target)
pred_bc = lr_bc.predict(boxcox_transformed)
r2_bc = r2_score(boxcox_target, pred_bc)
resid_bc = boxcox_target - pred_bc

print(f"R² (raw features → raw target):      {r2_raw:.4f}")
print(f"R² (Box-Cox features → Box-Cox target): {r2_bc:.4f}")

# Residual plots: raw vs Box-Cox
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

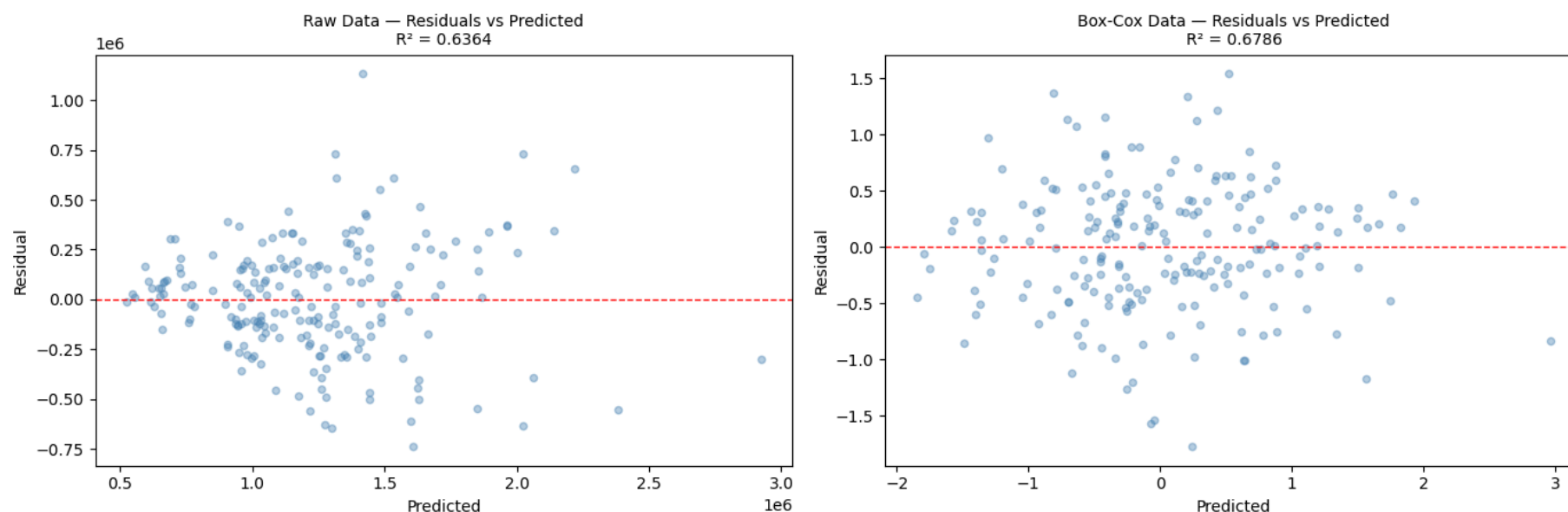
axes[0].scatter(pred_raw, resid_raw, alpha=0.4, s=20, color='steelblue')
axes[0].axhline(y=0, color='red', linestyle='--', linewidth=1)
axes[0].set_xlabel('Predicted')
axes[0].set_ylabel('Residual')
axes[0].set_title(f'Raw Data – Residuals vs Predicted\nR² = {r2_raw:.4f}', fontsize=10)

axes[1].scatter(pred_bc, resid_bc, alpha=0.4, s=20, color='steelblue')
axes[1].axhline(y=0, color='red', linestyle='--', linewidth=1)
axes[1].set_xlabel('Predicted')
axes[1].set_ylabel('Residual')
axes[1].set_title(f'Box-Cox Data – Residuals vs Predicted\nR² = {r2_bc:.4f}', fontsize=10)
```

```
plt.suptitle('Residual Diagnostics: Raw vs Box-Cox Transformed', fontsize=13, y=1.02)
plt.tight_layout()
plt.show()
```

R^2 (raw features $\rightarrow$ raw target): 0.6364
 R^2 (Box-Cox features $\rightarrow$ Box-Cox target): 0.6786

Residual Diagnostics: Raw vs Box-Cox Transformed



R^2 improves modestly from 0.6364 on the raw data to 0.6786 after Box-Cox transformation — a gain of roughly 4 percentage points. The improvement is not dramatic, which is consistent with the earlier finding that correlations are largely stable across transformation methods. The more informative comparison is in the residual plots. The raw data residuals show a clear fan shape: variance increases as predicted values grow, with residuals spreading from roughly $\pm 250,000$ at the low end to over $\pm 1,000,000$ at the high end. This is textbook heteroscedasticity and directly violates the constant-variance assumption of linear regression. After Box-Cox transformation, the residual spread is visibly more uniform across the range of predicted values — the fan shape is largely eliminated, with residuals distributed more evenly around zero. This confirms that Box-Cox transformation improves the suitability of the data for linear regression primarily by stabilising variance rather than by boosting predictive power.

6. Final Recommendation

Based on the evidence gathered through EDA and transformation analysis, the following transformation is recommended for each column before fitting a linear regression model to predict `median_house_price`.

```
In [14]: # Apply the final recommended transformation to all features and the target
# Box-Cox is applied to all columns – it corrects skewness where needed and
# standardises the output by default (zero mean, unit variance)

# Features – Box-Cox via PowerTransformer (already fitted in Section 4.5)
final_features = boxcox_transformed.copy()

# Target – Box-Cox
final_target = pd.Series(boxcox_target, index=raw.index, name=target)

# Final skewness and correlation summary
print("Final transformed dataset – skewness:")
print(final_features.skew().round(4).to_string())
print(f"\nTarget skewness after Box-Cox: {final_target.skew():.4f}")

print("\nFinal correlation with transformed target:")
print(final_features.corrwith(final_target).round(4).sort_values(ascending=False).to_string())

# Confirm scale consistency
print("\nFinal feature means and stds (should be ~0 and ~1):")
print(pd.DataFrame({
    'mean': final_features.mean().round(4),
    'std': final_features.std().round(4)
}).to_string())
```

Final transformed dataset – skewness:

```
number_of_houses    -0.0064
number_of_units     -0.0141
population           0.1005
aus_born_perc       -0.1286
median_income       -0.0084
```

Target skewness after Box-Cox: 0.0135

Final correlation with transformed target:

```
median_income        0.7151
aus_born_perc        0.3277
number_of_units      0.3197
number_of_houses     -0.0870
population           -0.2595
```

Final feature means and stds (should be ~0 and ~1):

```
              mean    std
number_of_houses -0.0000 1.0025
number_of_units  0.0000 1.0025
population       0.0000 1.0025
aus_born_perc    0.0000 1.0025
median_income    -0.0000 1.0025
```

6.1 Recommended Transformation per Column

Based on the evidence from Sections 3 through 5, the following transformations are recommended:

Column	Recommended Transformation	Justification
number_of_houses	Box-Cox ($\lambda=0.18$)	Heavily right-skewed (2.16). Log transformation corrected it to -0.44 but Box-Cox achieves near-perfect symmetry (-0.006). Scale is also resolved by Box-Cox's built-in standardisation.
number_of_units	Box-Cox ($\lambda=0.13$)	Most severely skewed feature (3.63). Log reduced this to -0.40; Box-Cox corrects it to -0.014. The near-zero lambda confirms log-like behaviour is appropriate but Box-Cox is more precise.
population	Box-Cox ($\lambda=0.55$)	Moderately right-skewed (1.09). Log transformation overcorrected to -2.65, introducing severe left skew. Box-Cox with $\lambda=0.55$ (approximately square root) corrects it cleanly to 0.10. Log is not appropriate for this column.

Column	Recommended Transformation	Justification
aus_born_perc	Box-Cox ($\lambda=2.46$)	Mildly left-skewed (-0.57) and near-symmetric. Shape correction is not strictly necessary, but Box-Cox reduces skewness to -0.13 and standardises the scale simultaneously, making it the efficient single-step choice.
median_income	Box-Cox ($\lambda=0.73$)	Near-symmetric (0.17) — no meaningful shape correction required. Box-Cox brings skewness to -0.008 and standardises the scale. Standardisation alone would have been sufficient here, but Box-Cox is applied for consistency.
median_house_price (target)	Box-Cox	Moderately right-skewed (1.03). The residual diagnostics in Section 5.1 showed clear heteroscedasticity (fan-shaped residuals) when the raw target was used, which was largely eliminated after Box-Cox transformation. Transforming the target is warranted because it stabilises residual variance and improves compliance with the homoscedasticity assumption. Box-Cox achieves near-zero skewness (0.014) and is preferred over log for the same reason as population — it optimises λ rather than fixing it at 0. One trade-off of transforming the target is that model predictions will be on the Box-Cox scale and must be back-transformed (via the inverse Box-Cox function) to recover predictions in the original dollar units.

Summary: Box-Cox transformation is applied uniformly to all five features and the target. It simultaneously addresses both criteria — scale disparity (via built-in standardisation to zero mean and unit variance) and distributional skewness (via optimised lambda per column). Standardisation and min-max normalisation were ruled out because they cannot correct skewness. Log transformation was ruled out as the sole method because it overcorrects `population`. Box-Cox subsumes both scaling and shape correction in a single step, making it the most appropriate preparation for linear regression on this dataset. Transforming the target is recommended because the residual diagnostics confirmed that an untransformed target produces heteroscedastic residuals; predictions from the final model will require inverse transformation to return to the original scale.

7. Conclusion

This notebook explored the effect of five transformation methods — standardisation, min-max normalisation, log transformation, power (square root) transformation, and Box-Cox transformation — on five features and the target `median_house_price`, with the goal of preparing the data for a linear regression model.

EDA revealed two problems in the raw data: severe scale disparity across features (population ranging to 54,005 versus `aus_born_perc` ranging to 88) and right skewness in four of the six columns of interest. Standardisation and min-max normalisation resolved scale

disparity but left skewness entirely unchanged, as both are linear transformations. Log transformation corrected skewness in `number_of_houses` and `number_of_units` but overcorrected `population` from 1.09 to -2.65. Square root transformation corrected population well (-0.03) but undercorrected `number_of_units` (1.32). Box-Cox transformation resolved both problems simultaneously — achieving near-zero skewness across all columns (within ± 0.13) while standardising the output to zero mean and unit variance by default.

The final recommended transformation is Box-Cox applied uniformly to all five features and the target. No additional scaling step is required. The transformed dataset satisfies both criteria for linear regression readiness: features are on a consistent scale and distributional skewness has been corrected across all columns.

Overall, Box-Cox transformation produced the most statistically suitable feature set for linear regression modelling because it simultaneously corrected distributional skewness, stabilised residual variance, and resolved scale disparity — addressing all three preparatory criteria in a single step.